

SYSTEM DESIGN · COMPLETE NOTES

Event-Driven Architecture

A comprehensive guide with diagrams, examples & interview tips

8

CORE TOPICS

4

COMPONENTS

3

DELIVERY SEMANTICS

3

PROCESSING PATTERNS

Prepared from lecture by **May** | Topic: Event-Driven Architecture (EDA) | System Design Series

TABLE OF CONTENTS

What's Inside

- | | | | |
|---|---------------------------------------|---|------------------------------------|
| 1 | Traditional Synchronous Architecture | 2 | What is Event-Driven Architecture? |
| 3 | What is an Event? | 4 | Four Core Components of EDA |
| 5 | Topics in EDA | 6 | Event Processing Patterns |
| 7 | Failure Handling & Delivery Semantics | 8 | EDA in Real-World Systems |



Traditional Synchronous Architecture

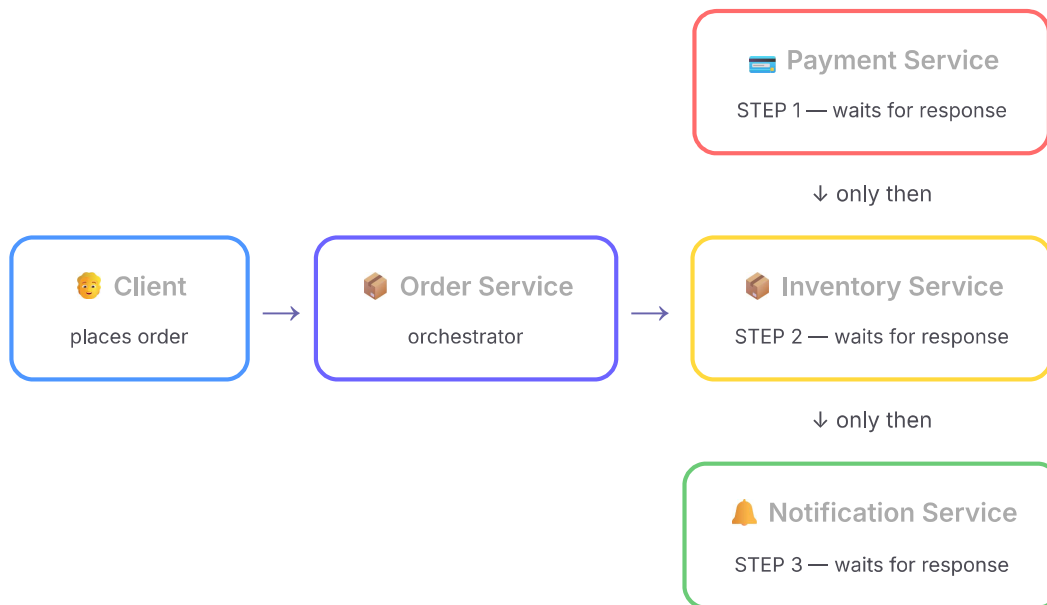
Understanding the problem that EDA was designed to solve

Before diving into Event-Driven Architecture (EDA), it's essential to understand the **traditional synchronous model** and its limitations. Only by seeing the problem clearly can we appreciate EDA as the solution.

How it works

In the traditional synchronous model, a client sends a request to a central service (e.g., *Order Service*), which then **calls each dependent service one by one and waits for each response** before moving to the next step.

DIAGRAM 1 — TRADITIONAL SYNCHRONOUS REQUEST FLOW



The Three Major Problems



1. Tight Coupling



2. Cascading Failure

Order Service must know the exact URL, request format, and response format of every dependent service. Adding or changing any service requires modifying Order Service code directly.

High Risk

If Payment Service goes down, the entire chain breaks. Inventory and Notification can never be called. The entire system appears offline even though only ONE service failed.

Critical

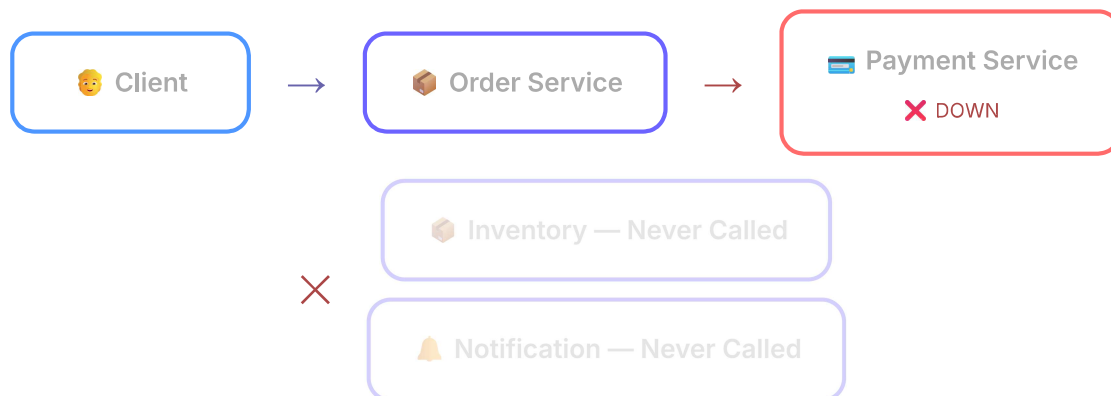


3. Hard to Extend

Adding a new Analytics service requires refactoring Order Service, re-testing everything, and taking it offline temporarily. Each new integration is costly and risky.

Medium Risk

DIAGRAM 2 — CASCADING FAILURE: ONE SERVICE DOWN = SYSTEM APPEARS DEAD



⚠️ Only Payment Service is down, but the entire ordering system fails for the user.

**Interview Tip**

When asked "What are the drawbacks of synchronous microservices?", always mention: tight coupling, cascading failures, and poor extensibility. These are the exact reasons EDA was invented.



What is Event-Driven Architecture?

The design principle that decouples services through events



Definition: Event-Driven Architecture (EDA) is a *software design principle* in which system components communicate by either **producing an event** or **responding to an event**. No service calls another directly.

Instead of Service A directly calling Service B and waiting for a reply, Service A **publishes an event** describing what happened. Any service that cares about that event **subscribes** to it and reacts independently. The key word is **asynchronous** — producers don't wait for consumers.

EDA vs. Synchronous: Side-by-Side



Synchronous (Old Way)

- Service A calls Service B directly
- A waits for B to respond before continuing
- Tight URL and format coupling between services
- One failure cascades through the system
- Hard to add new services without refactoring

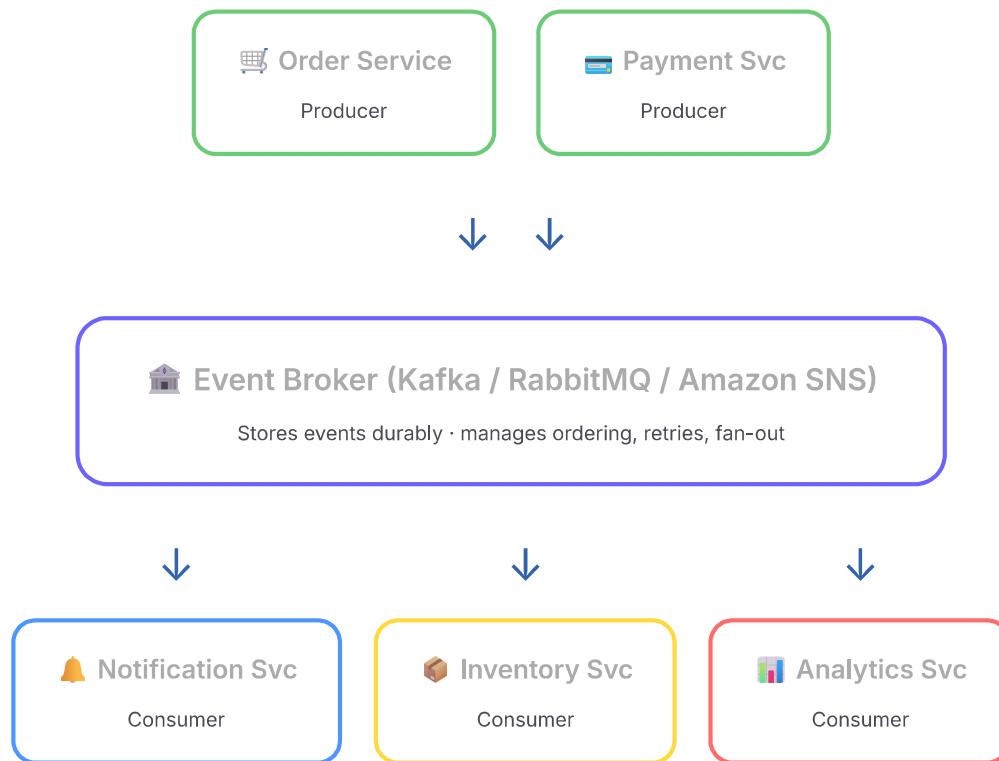


Event-Driven (New Way)

- Service A publishes an event to broker
- A moves on immediately — no waiting

- Zero direct coupling between services
- A single failure is isolated, others continue
- Add new consumers without touching producers

DIAGRAM 3 — EDA OVERVIEW: PRODUCERS → BROKER → CONSUMERS



Producers emit events and forget. Consumers subscribe and react independently. The broker holds everything together.



Key Insight — All Three Problems Solved

Tight coupling is eliminated (no direct calls). Cascading failures are prevented (each service is independent). New services are added by simply subscribing to an existing topic — no changes to producers needed.



What is an Event?

03

The fundamental unit of communication in EDA

 **Definition:** An event is a **durable, immutable record of something that has already happened** in the past. It is a fact — not a command, not a request — it describes what occurred.

The most important word is *past tense*. An event says *"this happened"*, not *"please do this"*. For example, `"order.created"` means the order was already created — it is now permanent history that cannot be changed.

Four Core Properties of an Event



IMMUTABLE

Once written, an event cannot be changed or deleted. History cannot be altered. It is a permanent, unmodifiable record of what happened.



PAST TENSE

Events describe things that have already happened. `"order.created"` not `"create.order"`. This linguistic distinction is architecturally important.



DURABLE

Events are persisted to disk and can survive system crashes and restarts. They are not held only in volatile memory — they outlast the process that created them.



REPLAYABLE

Events can be replayed from any point in time. This enables disaster recovery, rebuilding derived state, new service bootstrapping, and full auditing.

Real-World Event Name Examples

order.created

payment.completed

inventory.reserved

user.registered

shipment.dispatched

cart.abandoned

payment.failed

order.cancelled

review.submitted

What Does an Event Payload Look Like?

A typical event is a lightweight structured payload (usually JSON) containing metadata and the relevant data about what changed:

```
{
  "eventId": "evt-7a3f9b", // unique ID for deduplication
  "eventType": "order.created", // WHAT happened (past tense)
  "timestamp": "2024-09-15T10:30:00Z", // WHEN it happened
  "source": "order-service", // WHO produced it
  "data": {
    "orderId": 12345,
    "userId": 9876,
    "amount": 499.99
  }
}
```



Event vs. Command vs. Query — Know the Difference

An *event* says "this happened" (past fact, no expectation of response). A *command* says "do this now" (imperative, expects execution). A *query* asks "what is the current state?" (read-only request). EDA is built exclusively around **events**.



Four Core Components of EDA

04

Event Producer · Event Broker · Event Consumer · Topic

A · Event Producer

An **Event Producer** is any service that *detects a state change* and emits an event describing what happened. It does **not** know or care who will consume that event.

How a Producer Works — Step by Step

- 1 A state change occurs in the producer's domain (e.g., user places an order, completing a payment).
- 2 The producer creates an event object with the event type, timestamp, and relevant data.
- 3 The producer publishes the event to the Event Broker's designated topic.
- 4 The producer **immediately moves on** — it does not wait for any response or acknowledgment from consumers.

Two Critical Rules of an Event Producer

Rule 1 — Unaware of Consumers

The Order Service doesn't know Payment, Inventory, or Notification services exist. It just fires the event into the broker. This is the core mechanism that eliminates tight coupling. You can add or remove consumers without the producer ever knowing.

Rule 2 — Does NOT Wait

After emitting the event, the producer is done. It doesn't wait for acknowledgment, for processing to complete, or for success/failure feedback from any downstream system. "Fire and forget" is the producer's mode of operation.

Real-World Example — Amazon Order

When you click "Buy Now" on Amazon, the **Order Service** creates the order record in its database and emits an `order.created` event. The Order Service then immediately shows you "Order Confirmed" — it doesn't call Payment or Notification services directly. They react on their own after receiving the event.

B · Event Broker

The **Event Broker** is the *heart* of EDA. It accepts events from producers, stores them durably, and delivers them to the right consumers. It manages ordering, retries, and fan-out automatically.

What the Broker Does



ACCEPT EVENTS

Receives events from producers and acknowledges receipt, allowing the producer to move on immediately.



STORE DURABLY

Persists events to disk so they survive broker restarts or crashes. An event received by the broker is never lost.



DELIVER EVENTS

Routes events to all subscribed consumers, even if they were temporarily offline — events are queued until the consumer reconnects.



MANAGE ORDERING

Ensures events are delivered in the correct sequence so consumers process them in the right order (e.g., created before shipped).



HANDLE RETRIES

If a consumer fails to acknowledge processing, the broker retries delivery automatically until success or a max retry limit is hit.



FAN-OUT

One published event is delivered to ALL subscribed consumers simultaneously — a single event triggers multiple independent reactions.

Popular Event Broker Technologies

TECHNOLOGY	TYPE	BEST FOR	KEY STRENGTH
Apache Kafka	Pull-based	High-throughput streaming, event sourcing	Log-based, highly durable, replayable from any offset
RabbitMQ	Push-based	Task queues, low-latency routing	Flexible routing with exchanges and bindings
Amazon SNS/SQS	Both	AWS-native, serverless architectures	Fully managed, auto-scaling, pay-per-use
Google Pub/Sub	Both	GCP-native workloads	Global, at-least-once by default, no infrastructure



One-Liner to Remember in Interviews

The Event Broker is a
persistent event log + guaranteed delivery engine

. Once an event enters the broker, it is never lost, and every subscribed consumer will receive it.

C · Event Consumer

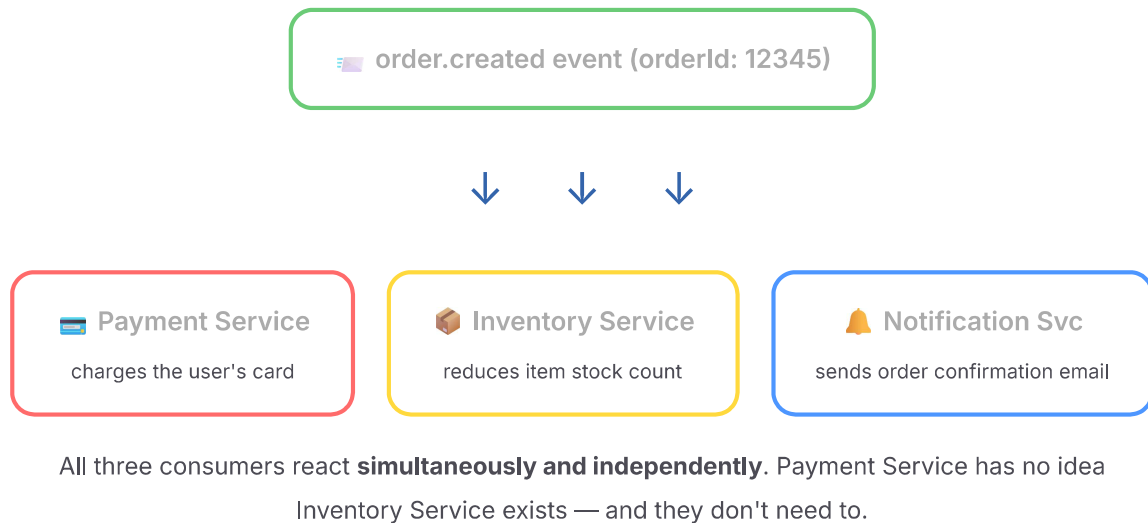
An **Event Consumer** is a long-running service that subscribes to one or more *topics* and reacts to incoming events. It acts completely independently of other consumers on the same topic.

How a Consumer Works — Step by Step

- 1 Consumer subscribes to a topic (e.g., "order-events") when the service starts up.

- 2 Consumer sits in a polling or listening loop, waiting for new events from the broker.
- 3 When an event arrives, the consumer processes it according to its domain logic (e.g., deducts payment, reduces stock).
- 4 Consumer sends an **acknowledgment (ACK)** to the broker once processing is fully complete and committed.
- 5 Consumer operates with zero knowledge of other consumers on the same topic.

DIAGRAM 4 — FAN-OUT: ONE "ORDER.CREATED" EVENT TRIGGERS THREE INDEPENDENT CONSUMERS



Critical Consumer Rule — Complete Isolation

Consumers do not know who the producer is. They do not know about other consumers on the same topic. Each consumer only knows: "I subscribed to this topic, I received an event, I will do my job." This complete isolation is the source of EDA's flexibility and resilience.



Topics in EDA

05

The append-only stream that organises related events

 **Definition:** A **Topic** is an append-only, ordered log of related events. Think of it as a named channel for a specific category of events. All events of the same type flow through the same topic.

Key Properties of a Topic



LOG FORMAT

Events are stored as sequential log entries, each identified by an offset number. Consumers track their position using an offset pointer.



APPEND-ONLY

You can only add new events to the end. You cannot modify or delete existing events. History is immutable and sacred.



SAME-TYPE EVENTS

All events in a topic belong to the same category or domain. "order-events" only holds order-related events — not payment events.



MULTI-CONSUMER

Multiple consumers can subscribe independently. Each maintains its own offset — one slow consumer doesn't block others from reading ahead.

Inside the "order-events" Topic — What Events Live There

DIAGRAM 5 — APPEND-ONLY LOG: THE "ORDER-EVENTS" TOPIC OVER TIME



Time flows left to right. Events are immutable, ordered, and identified by offset. A consumer reading from offset 0 will see all events in order.

Consumer Filtering Within a Topic

When a consumer subscribes to "order-events", it receives *all* event types in that topic. The consumer is responsible for **filtering** only the event types it cares about:

```
// Inventory Service subscribes to "order-events" topic
// It receives ALL events but only acts on "order.created"
```

```
consumer.subscribe("order-events");

onEvent(event) {
  if (event.type === "order.created") {
    reduceInventory(event.data.items); // do the work
  }
  // All other event types are ignored by this consumer
}
```

Common Topics in a Typical E-Commerce System

TOPIC NAME	EVENT TYPES INSIDE	WHO SUBSCRIBES
order-events	created, paid, packed, shipped, cancelled, returned	Payment, Inventory, Notification, Analytics
payment-events	initiated, completed, failed, refunded, disputed	Order Service, Fraud Detection, Analytics
user-events	registered, profile_updated, deleted, email_verified	Email Service, Recommendation, Auth Service
inventory-events	reserved, released, restocked, low_stock_alert	Purchasing, Notification, Order Service



Event Processing Patterns

06

Three common ways events carry data and how consumers react to them

Event processing patterns describe **how much data an event carries** and consequently how consumers retrieve and process that data. There are three fundamental patterns, each with distinct trade-offs.

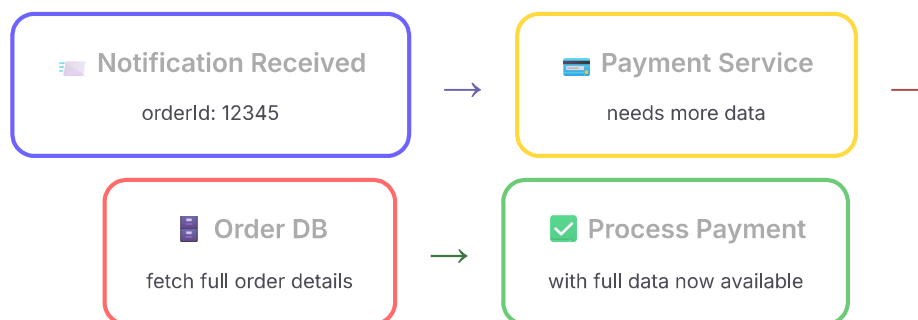
Pattern 1 — Event Notification

 The event carries only **minimal information** — just enough to notify consumers that something happened. Consumers must make additional calls to fetch the complete data they need to process the event.

Example Payload (Minimal)

```
{  
  "eventType": "order.created",  
  "orderId": 12345  
  // That's all! No amount, no userId, no items list.  
}
```

DIAGRAM 6 — EVENT NOTIFICATION: CONSUMER MUST MAKE EXTRA CALLS





Advantages

Small event payload means faster network transmission and lower broker storage costs.
Good when different consumers need very different subsets of data.

Lightweight



Disadvantages

Requires extra round-trip calls to databases or other services, increasing end-to-end latency.
Consumer is partially coupled to the producer's data source or API.

Extra DB Calls

Pattern 2 — Event-Carried State Transfer (ECST)

 The event carries **everything the consumer needs** to fully process it — completely self-contained. No additional calls to databases or other services are required.

Example Payload (Full Data)

```
{
  "eventType": "order.created",
  "orderId": 12345,
  "userId": 9876,
  "userEmail": "alice@example.com",
  "amount": 499.99,
  "currency": "USD",
  "paymentMethodId": "card_abc123",
  "items": [{"sku": "LAPTOP-X1", "qty": 1}],
  // Payment Service has everything – zero additional calls needed!
}
```



Advantages



Disadvantages

True decoupling — consumer is completely independent and needs zero additional data fetches. Lower end-to-end latency since no extra round trips are required.

Fully Decoupled

Larger event payloads consume more broker storage and network bandwidth. Risk of consumers processing stale data if the event sits in the queue for a long time before being processed.

Larger Payload

Pattern 3 — Event Sourcing

📖 Instead of storing the *current state* of an entity, the system stores a *complete sequence of all events* that led to that state. The current state is *derived* by replaying events — it is never stored directly.

Traditional Database vs. Event Sourcing

📖 Traditional DB — Stores State

When an order ships, the row is updated to "Shipped". All previous states are overwritten and permanently lost. You cannot answer "what was the status yesterday?"

```
orderId: 123
status: "Shipped" ← only this moment
// Previous states: gone forever
```

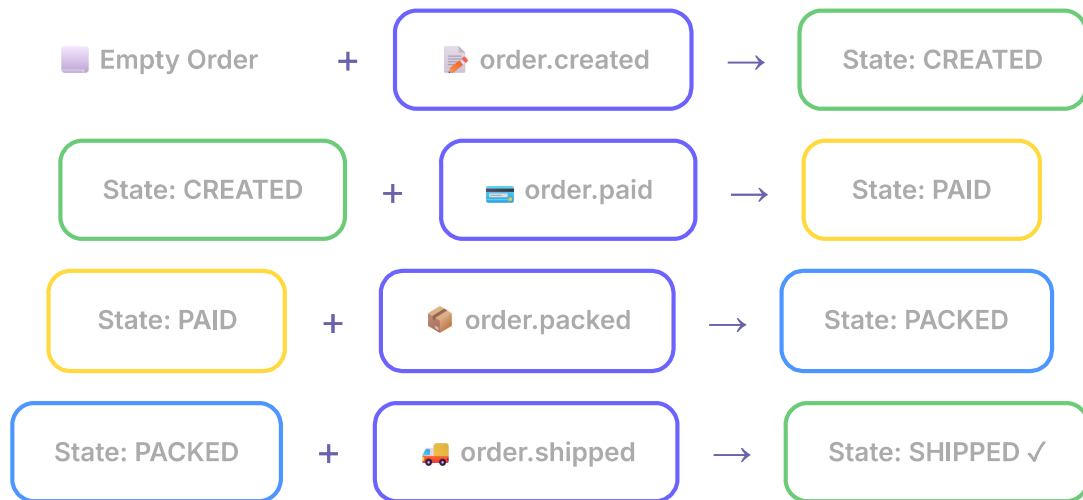
⚡ Event Sourcing — Stores Events

Every state change is recorded as a separate event. You replay them to derive any state at any point in time. Full history forever.

```
[0] order.created ← t=0
[1] order.paid ← t=1
```

```
[2] order.packed ← t=2  
[3] order.shipped ← t=3 ← full trail!
```

DIAGRAM 7 — EVENT SOURCING: STATE IS BUILT BY REPLAYING EVENTS IN SEQUENCE



By replaying events in order, you reconstruct the state at any point. Skip "order.paid" and you reach an incorrect state — ordering matters critically.



Advantages

Full audit trail — every state change is recorded. Time-travel debugging — reconstruct past states. Powers powerful analytics and compliance reporting.

Full Audit Trail



Disadvantages

Complex design — events must play in exact order. Storage grows indefinitely over time. Requires periodic "snapshots" to avoid replaying from the very beginning for performance.

Complex Design



Real-World Use Case: Banking & Finance

Banks use event sourcing for every single transaction. Your bank balance is not stored directly — it's the sum of all debit and credit events since account creation. This lets them answer "what was my balance at 2pm on Jan 1st?", trace fraud, and provide full regulatory compliance. All financial institutions rely on event sourcing.

All Three Patterns Compared

PROPERTY	EVENT NOTIFICATION	EVENT-CARRIED STATE	EVENT SOURCING
Event size	Small (minimal data)	Large (full data)	Small per event
Extra DB calls?	Yes — consumer fetches data	No — self-contained	No — replays events
Decoupling level	Medium	High	Very High
Historical audit?	None	None	Complete history
Design complexity	Low	Medium	High
Best use case	Simple triggers / webhooks	Independent microservices	Financial, compliance, CQRS



Failure Handling & Delivery Semantics

How EDA guarantees resilience when components fail



Failures Are Inevitable — But Isolated

EDA systems are prone to failures just like any distributed system. The key advantage is: EDA is designed so that a

single failure does NOT crash the whole system

. Each failure is isolated, and work resumes safely when the failed component recovers.

Types of Failures in EDA

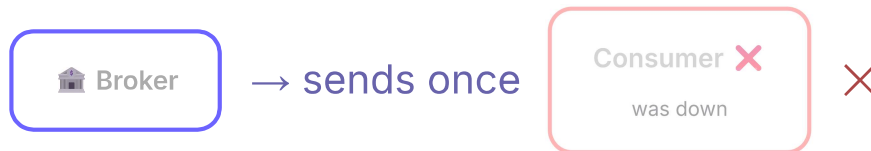
FAILURE TYPE	COMMON EXAMPLES	SYSTEM IMPACT
Consumer Failure	Process crash, unhandled exception during processing, downstream DB unavailable	Event not acknowledged; broker retries delivery to that consumer
Broker Failure	Node crash, temporary unavailability, partition leader re-election	Events buffered; replicated broker nodes continue serving traffic
Network Failure	TCP timeout, packet loss, temporary network partition	Retries at transport layer; idempotent consumers handle any resulting duplicates

Delivery Semantics — EDA's Answer to Failures

Delivery semantics define the guarantee the broker makes about how many times an event will be delivered to a consumer — even in the presence of failures. This is the critical mechanism that makes EDA resilient.

DIAGRAM 8 — THREE DELIVERY SEMANTICS: VISUAL COMPARISON OF GUARANTEES

1. At-Most-Once Delivery



Event LOST forever

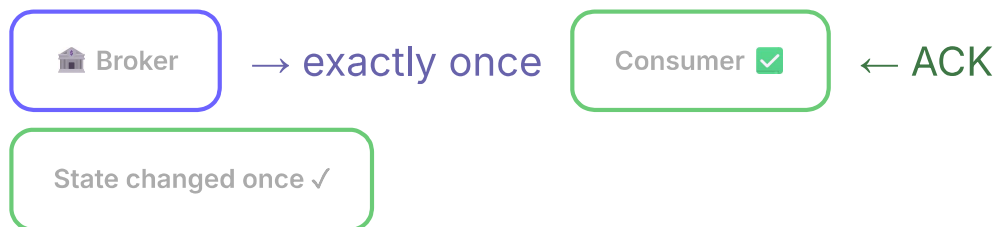
Broker sends event once and moves on — no ACK required, no retry. If consumer is down or crashes, the event is permanently lost. **Use when:** occasional data loss is acceptable (e.g., log metrics, monitoring heartbeats, analytics counters).

2. At-Least-Once Delivery (Default in EDA)



Broker retries delivery until it receives an ACK from the consumer. Event is **never lost**, but may arrive multiple times if the consumer fails after processing but before sending ACK. Consumers must be **idempotent**. **Use when:** data loss is unacceptable (most EDA use cases).

3. Exactly-Once Delivery



Each event affects the system state **exactly once** — even if the network fails and the broker retries. The broker uses distributed transactions or idempotency keys to ensure deduplication. Most complex to implement. **Use when:** absolute precision is required (e.g., financial settlements, wallet balance updates).

Idempotency — Making At-Least-Once Delivery Safe

With at-least-once delivery, the same event may arrive twice. The consumer must be **idempotent** — processing the same event twice produces the same final result as processing it once:

```
// ❌ Non-Idempotent (DANGEROUS with at-least-once)
function processPayment(event) {
  balance -= event.amount; // if called twice: double deduction!
}

// ✅ Idempotent (SAFE with at-least-once)
function processPayment(event) {
  if (processedEvents.has(event.eventId)) {
    return; // already processed – skip safely
  }
  balance -= event.amount;
  processedEvents.add(event.eventId); // mark as done
}
```

Delivery Semantics Comparison Table

PROPERTY	AT-MOST-ONCE	AT-LEAST-ONCE	EXACTLY-ONCE
Can event be lost?	Yes — possible	Never	Never
Can event be duplicated?	Never	Yes — possible	Never
Retries on consumer failure?	No	Yes, until ACK	Yes, with dedup
Needs idempotent consumer?	No	Yes — required	No (broker handles)
Implementation complexity	Low	Medium	High
Performance overhead	Lowest	Medium	Highest
Typical use case	Logs, metrics, monitoring	Most EDA systems (default)	Finance, payments, wallets



Default in EDA Systems

Event-driven systems use

At-Least-Once Delivery by default

(Kafka, RabbitMQ, SNS all default to this). This is why designing idempotent consumers is a non-negotiable skill in EDA system design — and a common interview question.



EDA in Real-World Systems

08

LinkedIn, Amazon, Netflix & Uber — how industry leaders use EDA

EDA is used in systems where a **single action must trigger multiple independent reactions reliably, at massive scale, without tight coupling between services**. If all three conditions apply, EDA is the right choice.

The Three Conditions for Choosing EDA



INDEPENDENT REACTIONS

Multiple services must react to the same event without depending on each other — each service acts autonomously on the event.



AT MASSIVE SCALE

The system must handle millions of events per second without performance degradation — EDA's async nature allows each service to scale independently.



NO TIGHT COUPLING

Services must evolve and deploy independently. Adding a new feature (new consumer) must not require changes to existing producers or other consumers.

Company Use Cases



LinkedIn

Powers its **Feed Service**, **Notification Service**, and **Recommendation Service**. When you post, all three services independently consume the "post.created" event and



Amazon

Builds its fault-tolerant, massively scalable **order lifecycle system** on EDA. Every stage — placed, paid, packed, shipped, delivered — is an event consumed by dozens of



Netflix

Powers its **recommendation engine**, **monitoring infrastructure**, and **analytics pipeline** using EDA. When you watch a video, events flow to recommendation, billing,

process it without knowing about each other.

downstream services independently.

content analytics, and A/B testing systems independently.

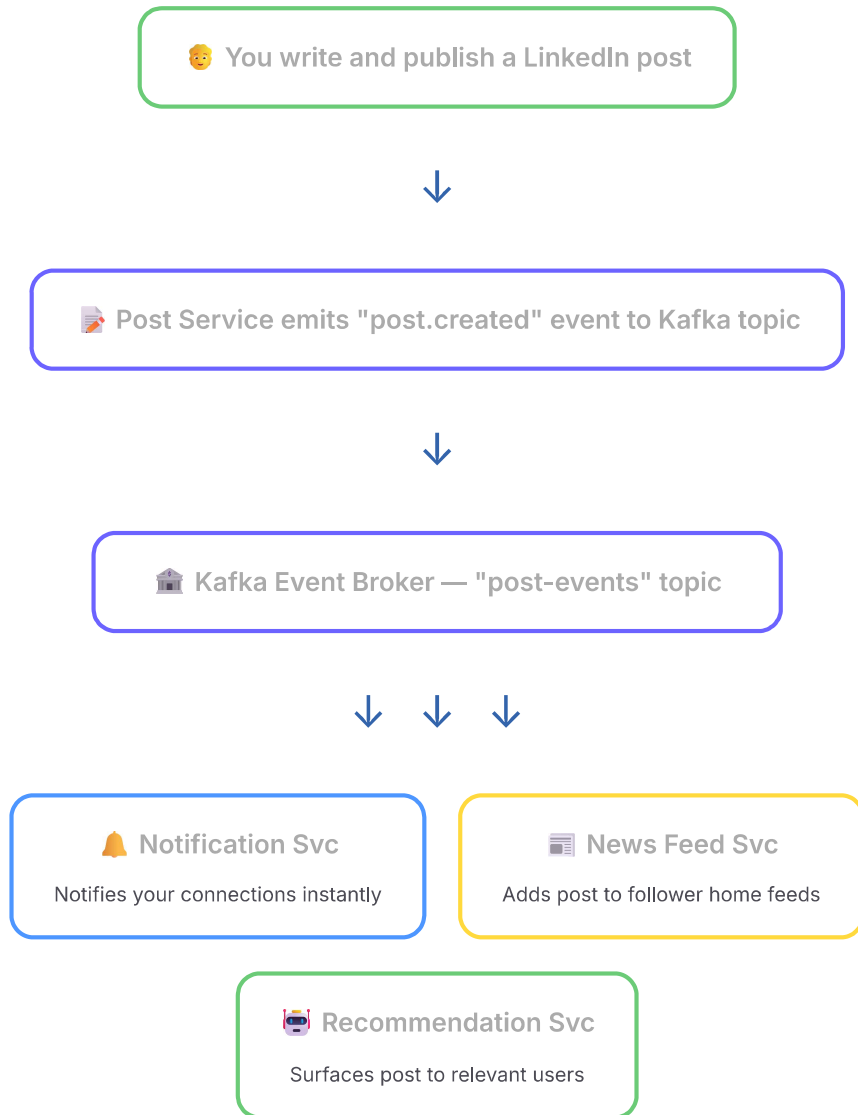


Uber

Uses EDA for the entire **ride lifecycle**:
ride.requested →
driver.assigned →
ride.started →
ride.completed →
payment.processed. Each event triggers pricing, mapping, driver scoring, and notification services independently.

Deep Dive — LinkedIn Post Creation Event Flow

DIAGRAM 9 — LINKEDIN: ONE POST CREATION TRIGGERS THREE INDEPENDENT SERVICES



Adding a 4th service (e.g., "Trending Topics Analyzer") requires zero changes to the Post Service or any existing consumers. Simply subscribe to the topic.

When NOT to Use EDA

**EDA is not always the right tool**

Avoid EDA when: (1) you need a **synchronous response**

(e.g., login validation — you must know immediately if credentials are valid), (2) your system is

small and simple

with only 2-3 services where the overhead isn't justified, (3)

event ordering

is so complex that maintaining it would be more costly than synchronous calls, or

(4) your team lacks expertise in distributed messaging systems and the operational overhead would be prohibitive.

Complete EDA Cheat-Sheet — All Concepts at a Glance

CONCEPT	ONE-LINE DEFINITION	KEY RULE OR EXAMPLE
Event	Immutable record of a past occurrence	"order.created" — fact that cannot be changed
Event Producer	Detects state change, emits event, forgets immediately	Order Service emits and doesn't wait for response
Event Broker	Heart of EDA — accepts, stores, and delivers events	Kafka, RabbitMQ, Amazon SNS/SQS
Event Consumer	Subscribes to topic, reacts to events independently	Payment Service processes order.created autonomously
Topic	Append-only log of same-category events	"order-events" holds created, paid, shipped events
Event Notification	Minimal payload — consumer fetches remaining data	Smaller event, but requires extra DB calls
Event-Carried State	Full payload — consumer is completely self-sufficient	Larger event, but fully decoupled consumer
Event Sourcing	Store events not state; replay to derive current state	Banks use this for full transaction audit trail
At-Most-Once	Send once; may be lost; never duplicated	OK for log metrics and monitoring
At-Least-Once	Never lost; may be duplicated; retry until ACK	Default in EDA; requires idempotent consumers
Exactly-Once	Delivered and processed precisely once; no loss no dup	Required for financial settlements and wallets

CONCEPT**ONE-LINE DEFINITION****KEY RULE OR EXAMPLE****Idempotency**

Processing same event twice =
same final result as once

Use eventId to detect and skip
duplicates

 Event-Driven Architecture — Complete Study Notes

Based on System Design lecture by **May** · Formatted for study & interview preparation

💡 To save as PDF: Click "Save as PDF" button → or press **Ctrl+P** → Set destination to **"Save as PDF"** → Adjust margins to Minimum → Save